

Motor de Inferencia Lógica como Servicio

Reporte Técnico del Proyecto

Materia: Paradigmas de Programación

1. Introducción

El presente proyecto implementa un Motor de Inferencia Lógica como Servicio (Logic Inference Engine as a Service). El sistema expone una API REST desarrollada en Node.js con Express que permite ejecutar consultas sobre una base de conocimiento escrita en Prolog a través del motor Tau Prolog.

El sistema actúa como un motor de inferencia simbólica: el usuario envía consultas lógicas en formato Prolog y recibe, de manera automática, resultados derivados mediante reglas declarativas. El dominio de aplicación elegido es la gestión de contratos empresariales, donde se infieren condiciones como penalizaciones, auditorías, riesgo y renovación de contratos.

Los principales objetivos del proyecto son:

- Demostrar la interacción entre paradigmas de programación: lógico, funcional y asíncrono.
- Construir un servicio real y ejecutable que combine Node.js con un motor Prolog embebido.
- Aplicar inferencia automática para derivar conocimiento a partir de hechos y reglas declarativas.

2. Explicación de Paradigmas de Programación

2.1 Programación Lógica (Prolog / Tau Prolog)

La programación lógica se basa en la definición de hechos y reglas para que el motor de inferencia derive conclusiones automáticamente mediante unificación y backtracking. En este proyecto la base de conocimiento (base.pl) contiene:

- Hechos: afirmaciones atómicas sobre el dominio, como `contract(contract1, alice, software_dev, 12, 50000)`.
- Reglas: definiciones lógicas que infieren nuevas verdades a partir de hechos.

Ejemplo de regla en el proyecto:

```
penalty_applicable(ContractID) :-  
    contract(ContractID, _, _, _, _),  
    delivered_on_time(ContractID, false),  
    quality_approved(ContractID, false).
```

Tau Prolog es una implementación de Prolog en JavaScript que se ejecuta dentro del proceso Node.js, permitiendo integrar el motor lógico directamente en el servidor sin dependencias externas adicionales. El motor recibe la consulta, la evalúa contra la base de conocimiento y retorna todas las soluciones posibles mediante backtracking.

2.2 Programación Funcional (JavaScript)

La programación funcional enfatiza el uso de funciones puras (sin efectos secundarios), la composición de funciones y la inmutabilidad de datos. En el proyecto se aplica en la capa de transformación de datos:

- `normalizeQuery`: función pura que asegura que la consulta tenga punto final y sin espacios extra.
- `pipe`: composición funcional que aplica una lista de funciones en secuencia sobre un valor.
- `prepareQuery`: pipeline construido con `pipe` que normaliza la entrada del usuario.
- `formatSolution`: transforma el objeto de sustituciones de Tau Prolog a un objeto plano serializable.

Ejemplo de composición funcional en el servidor:

```
const pipe = (...fns) => (x) => fns.reduce((v, f) => f(v), x);

const prepareQuery = pipe(
  (q) => (typeof q === "string" ? q : String(q)),
  (q) => q.replace(/\r\n/g, " "),
  normalizeQuery
);
```

Este estilo garantiza que cada transformación sea predecible, testeable de forma aislada y sin efectos secundarios, características propias del paradigma funcional.

2.3 Programación Asíncrona (Node.js)

Node.js utiliza un modelo de ejecución basado en un event loop de un solo hilo con I/O no bloqueante. Esto permite manejar múltiples solicitudes concurrentes sin necesidad de crear un hilo por cada petición. En el proyecto la asincronía se aplica en tres puntos clave:

- `loadKnowledgeBase`: lectura asíncrona del archivo `base.pl` con `fs.promises.readFile` (no bloquea el servidor).
- `createSession`: creación de la sesión Tau Prolog envuelta en una Promise, permitiendo `await`.
- `runQuery`: recolección de soluciones mediante recursión asíncrona con Promises; cada llamada a `answer()` es no bloqueante.

Ejemplo de función asíncrona en el servidor:

```
const loadKnowledgeBase = async () => {
  const kbPath = path.join(__dirname, "..", "knowledge", "base.pl");
  return fs.readFile(kbPath, "utf-8");
};
```

El endpoint `POST /query` usa `async/await` para orquestar la carga de la KB, creación de sesión e inferencia, todo de forma no bloqueante, lo que permite que el servidor atienda otras solicitudes mientras procesa cada consulta.

3. Arquitectura del Sistema

El sistema está compuesto por tres capas integradas en un único módulo Node.js:

Componente	Tecnología	Paradigma
Servidor HTTP	Express.js	Asíncrono
Motor de inferencia	Tau Prolog	Lógico
Base de conocimiento	Prolog (.pl)	Lógico
Transformación de datos	Funciones puras JS	Funcional
Gestión de I/O	async/await + Promises	Asíncrono

3.1 Flujo de Ejecución

Cuando el cliente realiza una solicitud POST /query, el sistema ejecuta el siguiente flujo:

- 1. El cliente envía un JSON con el campo query (consulta Prolog).
- 2. El servidor aplica el pipeline funcional para normalizar la consulta.
- 3. Se lee la base de conocimiento de forma asíncrona desde disco.
- 4. Se crea una sesión Tau Prolog y se carga el programa (await).
- 5. El motor lógico evalúa la consulta y recolecta todas las soluciones.
- 6. El servidor retorna las soluciones como respuesta JSON.

3.2 Endpoints de la API

Método	Ruta	Descripción
POST	/query	Ejecutar consulta Prolog — retorna soluciones JSON
GET	/health	Verificar estado del servicio
GET	/examples	Obtener consultas de ejemplo predefinidas

3.3 Estructura de Archivos

```
motor-inferencia-logica/  
├─ src/  
│   └─ server.js           # Servidor Express + lógica asíncrona y funcional  
├─ knowledge/  
│   └─ base.pl             # Base de conocimiento Prolog  
└─ scripts/
```

```
| └─ build.js           # Script para generar ejecutables
| └─ package.json
| └─ README.md
```

4. Ejemplo de Ejecución

4.1 Iniciar el servicio

```
npm install
npm start
# → 🌀 Motor de Inferencia Lógica escuchando en http://localhost:3000
```

4.2 Consulta: contratos con penalización aplicable

Solicitud HTTP:

```
POST http://localhost:3000/query
Content-Type: application/json

{ "query": "penalty_applicable(X)." }
```

Respuesta JSON:

```
{
  "success": true,
  "query": "penalty_applicable(X).",
  "solutionsCount": 3,
  "truncated": false,
  "solutions": [
    { "X": "contract1" },
    { "X": "contract3" },
    { "X": "contract4" }
  ],
  "elapsedMs": 14
}
```

El motor infirió que contract1, contract3 y contract4 tienen penalización aplicable porque ambos incumplieron la entrega a tiempo y no aprobaron el control de calidad, según las reglas definidas en base.pl.

4.3 Consulta: resumen completo de un contrato

```
{ "query": "contract_summary(contract3,C,T,D,V,P,S,H)." }
```

Respuesta:

```
{
```

```
"solutions": [{  
  "C": "charlie", "T": "maintenance",  
  "D": "24", "V": "80000",  
  "P": "yes", "S": "no", "H": "yes"  
}]  
}
```

El motor derivó que contract3 tiene penalización (P=yes), no es exitoso (S=no) y es de alto valor (H=yes), todo inferido automáticamente a partir de los hechos base sin lógica imperativa.

4.4 Contratos que requieren auditoría

```
{ "query": "audit_required(X)." }
```

Respuesta: contract1, contract3, contract4 — derivados por las reglas audit_required/1 que encadenan penalty_applicable y at_risk_contract.

5. Conclusiones y Extensiones

5.1 Conclusiones

El proyecto demostró con éxito la integración de tres paradigmas de programación en un sistema cohesionado y funcional:

- La programación lógica (Tau Prolog) permite definir el conocimiento del dominio de forma declarativa, separando qué se sabe de cómo se computa, lo que facilita la mantenibilidad y extensibilidad de las reglas.
- La programación funcional aporta transparencia y predecibilidad en la capa de transformación de datos, haciendo que el código sea más testeable y libre de efectos secundarios inesperados.
- La programación asíncrona de Node.js permite que el servicio maneje múltiples consultas concurrentes sin bloquear, lo que es esencial para un servicio web en producción.

La combinación de estos tres paradigmas produce un sistema donde cada parte está expresada en el estilo más apropiado para su naturaleza: el conocimiento como lógica, las transformaciones como funciones puras y la concurrencia como asincronía.

5.2 Posibles Extensiones

Extensión	Descripción
Base de conocimiento dinámica	Permitir agregar hechos y reglas via API REST (assert/retract) sin reiniciar el servicio.

Extensión	Descripción
Persistencia	Almacenar la KB en una base de datos para que los cambios sobrevivan reinicios del servidor.
Autenticación	Agregar JWT o API keys para controlar el acceso a los endpoints de consulta.
Frontend web	Interfaz gráfica para construir y ejecutar consultas Prolog de forma visual.
Streaming de soluciones	Usar Server-Sent Events para retornar soluciones incrementalmente en consultas largas.
Múltiples KB	Soportar varias bases de conocimiento y permitir seleccionar cuál usar en cada consulta.
Caché de sesiones	Cachear sesiones Prolog ya compiladas para mejorar el rendimiento en consultas repetidas.
Tracing lógico	Exponer la traza de inferencia (pasos de backtracking) para depuración y auditoría.

En conjunto, el proyecto sienta las bases para un sistema de razonamiento simbólico como servicio, aplicable a dominios como cumplimiento normativo, diagnóstico médico asistido, sistemas expertos legales o cualquier área donde el conocimiento pueda expresarse mediante reglas declarativas.